

Installing Ubuntu 24.04 with Btrfs on root



ferrario.me/ubuntu-on-the-raspberry-pi-part-i-installing-ubuntu-24.04-with-btrfs-on-root

Agustin Ferrario

September 5, 2024

I have been using Btrfs as the main (and only) filesystem on my Laptop for some time now. Since I like using it, I decided to use it also on my Raspberry Pi running Ubuntu Server 24.04 LTS. Here is the guide on how I did it.

This is the first of my series on Ubuntu on the Raspberry Pi. You could also use this guide for other distros, probably with some minor changes. The next two articles will talk about the use of [cloud-init](#) for customizing your system; and how to recover using btrfs snapshots.

Why?

Btrfs has a couple of things that I really like. First of all, compression is great, it helps me save space and reduces the I/O to my disks. This is great for flash memory, such as SSDs, USB Sticks and SD Cards, since it helps to reduce the wear and tear at the cost of some CPU. However, with [Zstandard compression](#) (zstd), the penalty is unnoticeable.

Snapshots are also an awesome feature of Btrfs. I can create a snapshot before an update and rollback if my system breaks. I also take a snapshot of my container data before upgrading my containers. This has saved my bacon a couple of times, so it's a nice thing to have.

Some people would say that btrfs is not stable enough because of bad experiences. However, I'm not using raid, so I cannot judge about it. In my experience and my use case, Btrfs is stable and has been in the mainline Linux Kernel for a long time.

Finally, the main reason I use it is because I can and because I have fun doing it.

If you don't like Btrfs and you would rather use zfs, I think that's a great idea, and you should check this guide from the [OpenZFS Documentation](#)

Let's get to Business

I will use a Linux system to set this up. So I will assume that you are using one, and you have some familiarity with it.

I will be using **/dev/sdX** for the USB-SSD. Please use change according to your situation.

I say USB-SSD, because it's a popular medium to use with a Pi, however you could use any USB attached storage that you want with it: USB-HDD, USB-Stick, etc.

Get the image and decompress it

Download the official 24.04 server image from the Ubuntu website:

<https://ubuntu.com/download/raspberry-pi>. After that, uncompress it to your favourite directory. Remember that xz will delete the file after decompressing. If you want to keep the compressed image, use the flag `--keep`.

```
xz --decompress ubuntu-24.04-preinstalled-server-arm64+raspi.img.xz
```

If everything worked correctly, you should now have a file called `ubuntu-24.04-preinstalled-server-arm64+raspi.img` on your folder.

Now we can copy it to our drive:

This is the point of no return. All the information on the drive **will be lost**. Please *make sure* that you wrote the right letter on the `of` part of `dd`.

```
dd if=ubuntu-24.04-preinstalled-server-arm64+raspi.img of=/dev/sdX status=progress
```

After that, our disk will have two partitions:

- A 512 MiB fat32 partition with the label `system-boot`.
- A 2.85 GiB ext4 partition with the label `writable`.

The last one, we would get rid of in the next step.

Preparing the disk

Now let's prepare our USB-Disk for our installation. Now we will delete the ext4 partition and create a new Btrfs root partition. You could convert the partition from ext4 to btrfs, however we would need to rearrange the data into different subvolumes after the fact, and I find it easier this way.

I will be using the command line (because it's the same for all Linux distros), but you can use a GUI tool such as [gparted](#) or [partition manager](#).

```
fdisk /dev/sdX
```

```
CCommand (m for help): d
```

```
Partition number (1,2, default 2):
```

```
Partition 2 has been deleted.
```

```
Command (m for help): n
```

```
Partition type
```

```
  p   primary (1 primary, 0 extended, 3 free)
```

```
  e   extended (container for logical partitions)
```

```
Select (default p):
```

Using default response p.
Partition number (2-4, default 2):
First sector (1050624-500118191, default 1050624):
Last sector, +/-sectors or +/-size{K,M,G,T,P} (1050624-500118191, default 500118191):

Created a new partition 2 of type 'Linux' and of size 238 GiB.
Partition #2 contains a ext4 signature.

Do you want to remove the signature? [Y]es/[N]o: y

The signature will be removed by a write command.

Command (m for help): w
The partition table has been altered.
Calling ioctl() to re-read partition table.
Synching disks.
mkfs.btrfs -L btrfs-root /dev/sdX2

The first command opens a partitioning tool called fdisk; deletes the last partition and then creates a new one using all the space on the disk.

The second command will format our partition to Btrfs and assign the label “btrfs-root” to it. The label will be useful afterwards.

Mounting our partitions

Now let's mount the partitions. We will be mounting our Btrfs partition with the compression flag, so all files are compressed when writing them to the disk.

First, we create some mount points:

```
mkdir /mnt/system-boot  
mkdir /mnt/new-root
```

Then we mount our partitions :

```
mount /dev/disk/by-label/system-boot /mnt/system-boot/  
mount /dev/disk/by-label/btrfs-root -o defaults,noatime,compress=zstd:1,autodefrag  
/mnt/new-root/
```

For understanding these mount options, we can check the explanations from the Btrfs [documentation](#) ¹:

Mount option	Explanation
defaults	Activate the defaults options
noatime	Do not update inode access times on this filesystem. ²

Mount option	Explanation
compress=zstd:1	Enable zstd compression and specify the level to 1, since we don't want to overtax our CPU
autodefrag	Enable automatic file defragmentation. This happens online. Recommended for HDDs 3

We would also need to mount the `writable` partition from the original image, to copy those files into our new Btrfs partition. We deleted the writable partition in our disk, however we can use a tool called `kpartx` to mount partitions from the image file.

```
kpartx -a ubuntu-24.04-preinstalled-server-arm64+raspi.img
mkdir /mnt/old-root
mount /dev/disk/by-label/writable /mnt/old-root/
```

Make sure to mount `system-boot` before you run `kpartx`, because running the command will create another disk with the same label.

Creating the subvolumes

We will create subvolumes for different applications. This allows us to avoid creating snapshots of unnecessary data (such as cache) and/or avoid the loss of data when doing a rollback. This was inspired by the [openSUSE Wiki](#), Didier Roche blogpost about [ZSys datasets layout](#) and the [OpenZFS Documentation](#):

Subvolumes	Mount point	Observations
@	/	The root subvolume with all the information from our OS
@home	/home	Home subvolume, so it gets its own snapshots
@root	/root	Home directory for the root user, so it's preserved on a rollback
@swap	/swap	Extra subvolume for the swapfile (OPTIONAL) 4
@usr_local	/usr/local	Manually compiled packages will be installed here
@tmp, @var_tmp	/tmp, /var/tmp	Temporary files, we don't need a snapshot of this one

Subvolumes	Mount point	Observations
@var_log	/var/log	Log files, so they are preserved on a rollback ⁵
@var_mail, @var_spool	/var/mail, /var/spool	Directories containing mail and/or mail queues ⁶
@var_snap	/var/snap	Snap can handle their own versioning
@snapshots	/.snapshots	Snapshots directories for Snapper

Creating different subvolumes, instead of creating them all inside the root subvolume, allows us to delete it if we need it. If you need an example, when this can be needed, wait for the third part of this series, when we talk about **Disaster Recovery**.

So now that we know what subvolumes we want, let's create them:

```
btrfs subvolume create /mnt/new-root/\@
btrfs subvolume create /mnt/new-root/\@home
btrfs subvolume create /mnt/new-root/\@root
btrfs subvolume create /mnt/new-root/\@swap
btrfs subvolume create /mnt/new-root/\@usr_local
btrfs subvolume create /mnt/new-root/\@tmp
btrfs subvolume create /mnt/new-root/\@var_tmp
btrfs subvolume create /mnt/new-root/\@var_log
btrfs subvolume create /mnt/new-root/\@var_mail
btrfs subvolume create /mnt/new-root/\@var_spool
btrfs subvolume create /mnt/new-root/\@var_snap
btrfs subvolume create /mnt/new-root/\@snapshots
```

There are some other directories that should have their own subvolume to avoid data loss. Here are some examples for important ones, you would want to create the ones you need:

Subvolumes	Observations
Databases (on /var/lib/)	Directories where the data for our databases is stored ⁷
/var/lib/named	DNS Server data
/srv and /var/www	FTP and Web Server Storage
/var/lib/docker/	Data for the docker daemon. For example, our container volumes. ⁸
/opt/docker	This one is mine. Here I store my Compose Files and container data

You will probably also want to disable compression and Copy-on-Write for some of these directories (specially with databases). Check the section about the [Swapfile](#) for more information on how to do it.

So after you decided which subvolumes you need, create them. For example, the docker ones:

```
btrfs subvolume create /mnt/new-root/\@docker
btrfs subvolume create /mnt/new-root/\@var_lib_docker
```

After we created the subvolumes, we need to remount them in the proper locations. For doing this, we will need to create mount points inside the root. We will specify the compression and autodefrag only on the first mounted subvolume⁹:

```
umount /mnt/new-root
mount /dev/disk/by-label/btrfs-root -o
defaults,noatime,compress=zstd:1,autodefrag,subvol=\@ /mnt/new-root

mkdir /mnt/new-
root/{home,root,swap,usr,usr/local,tmp,var,var/tmp,var/log,/var/snap,/var/mail,/var
/spool,.snapshots}
mkdir /mnt/new-root/{var/lib{,/docker},opt/{,/docker}}
mount /dev/disk/by-label/btrfs-root -o noatime,subvol=\@home /mnt/new-root/home/
mount /dev/disk/by-label/btrfs-root -o noatime,subvol=\@root /mnt/new-root/root/
mount /dev/disk/by-label/btrfs-root -o noatime,subvol=\@swap /mnt/new-root/swap/
mount /dev/disk/by-label/btrfs-root -o noatime,subvol=\@usr_local /mnt/new-
root/usr/local/
mount /dev/disk/by-label/btrfs-root -o noatime,subvol=\@tmp /mnt/new-root/tmp
mount /dev/disk/by-label/btrfs-root -o noatime,subvol=\@var_tmp /mnt/new-
root/var/tmp
mount /dev/disk/by-label/btrfs-root -o noatime,subvol=\@var_log /mnt/new-
root/var/log/
mount /dev/disk/by-label/btrfs-root -o noatime,subvol=\@var_mail /mnt/new-
root/var/mail/
mount /dev/disk/by-label/btrfs-root -o noatime,subvol=\@var_spool /mnt/new-
root/var/spool/
mount /dev/disk/by-label/btrfs-root -o noatime,subvol=\@var_snap /mnt/new-
root/var/snap/
mount /dev/disk/by-label/btrfs-root -o noatime,subvol=\@snapshots /mnt/new-
root/.snapshots/
```

I will also mount my optional ones, however in my case I won't actually need them:

```
mount /dev/disk/by-label/btrfs-root -o noatime,subvol=\@var_lib_docker /mnt/new-
root/var/lib/docker
mount /dev/disk/by-label/btrfs-root -o noatime,subvol=\@docker /mnt/new-
root/opt/docker/
```

And then, we check that everything was mounted properly:

```

mount | grep /mnt
/dev/sdX1 on /mnt/system-boot type vfat
(rw,relatime,fmask=0022,dmask=0022,codepage=437,ioccharset=iso8859-
1,shortname=mixed,errors=remount-ro)
/dev/mapper/loop0p2 on /mnt/old-root type ext4 (rw,relatime)
/dev/sdX2 on /mnt/new-root type btrfs
(rw,noatime,compress=zstd:1,space_cache=v2,autodefrag,subvolid=256,subvol=/@)
/dev/sdX2 on /mnt/new-root/home type btrfs
(rw,noatime,compress=zstd:1,space_cache=v2,autodefrag,subvolid=257,subvol=/@home)
/dev/sdX2 on /mnt/new-root/root type btrfs
(rw,noatime,compress=zstd:1,space_cache=v2,autodefrag,subvolid=258,subvol=/@root)
[...]

```

The places are the ones, that we specified, and all the flags (compression, autodefrag, noatime, etc.) are those, that we wanted, so we can continue to the next step.

Creating a swapfile (OPTIONAL)

By default, Ubuntu server on the Raspberry Pi doesn't use a swapfile or a swap partition. It's too much I/O for SD Cards, and it shortens the lifespan of your SSD or Flash Drive. However, if you don't have one, your system will most likely crash, if it encounters too much memory pressure and it's unable to kill processes in time. I would rather err on the side of caution.

You may also ask, "Why not use a swap partition instead of a file?", and the reasons for me are simple:

- I haven't seen a performance difference between a swapfile and a swap partition.
- I can easily remove the swapfile if I think that I don't need it any more.
- Swapfiles are supported on Btrfs since kernel 5.0 with some small caveats^{[10](#)}.
- It makes it easier to write this guide, because if you don't want a swapfile you can ignore this step. If you want a swap partition, you need to change the whole partitioning of the disk.

So, with that cleared, let's create our swapfile. If your host has btrfs-progs 6.1 or later, you can use:

```
btrfs filesystem mkswapfile --size 4G /mnt/new-root/swap/swapfile
```

Otherwise, use these steps:

```

touch /mnt/new-root/swap/swapfile
chattr +C /mnt/new-root/swap/swapfile
btrfs property set /mnt/new-root/swap/swapfile compression none
fallocate --length 4GiB /mnt/new-root/swap/swapfile
chmod 600 /mnt/new-root/swap/swapfile
mkswap /mnt/new-root/swap/swapfile

```

Do the second step for setting No Copy-on-Write to databases and other files. (You can specify the flag `-R`, for recursive)

In the next guide, we will set the swappiness to 20, instead of the default 60. This will help to avoid too much writing onto the swap.

Now that we have what we needed, we can copy the files into our new filesystem.

Copying the files into the new root

After we mounted everything in place, we need to copy all the files from the original image file to our Btrfs filesystem. We cannot copy and paste from the GUI because it could introduce many problems. For doing this, we use our well known `rsync`:

```
rsync --archive --verbose --hard-links --whole-file /mnt/old-root/ /mnt/new-root/
```

If you want the shorter version:

```
rsync -avHW /mnt/old-root/ /mnt/new-root/
```

If you want to understand, what each of these flags does, before blindly copying and pasting¹¹, you should check explainshell.com.

Modifying the fstab

Since we changed the basic structure of our file system, we require a new `fstab`, that specifies where each of the partitions and subvolumes should be mounted. In my [GitLab repo](#) you can find a well commented example. You should just download the repo (or copy and paste the parts that you need in a new file), modify it, and then copy it into your new-root.

You will at least need the part titled `Boot` partition and the `Root` filesystem one. Likewise, you should modify adding the mount points and subvolumes that you created. There is also an example for an external NTFS formatted disk¹².

If you need some help writing the `fstab`, you can check the articles about it in [How to geek](#) and [Red Hat Documentation](#), and of course the [man page](#) for `fstab`.

After you are satisfied with your new `fstab` we copy it to the new filesystem:

```
mv fstab.example /mnt/new-root/etc/fstab
```

Changing the cmdline arguments

The Raspberry Pi uses a file called `cmdline` to store the **Kernel Command Line** arguments. Since we changed the filesystem that our pi will boot on, we need to modify

these arguments:

```
root=LABEL=writable rootfstype=ext4
```

And change them for these:

```
root=LABEL=btrfs-root rootfstype=btrfs rootflags=subvol=@
```

What we did is change the LABEL of the partition, specify the filesystem type (rootfstype) and we supply the name of the subvolume that is used for the root filesystem.

If you want, you can just use the cmdline in my [repo¹³](#):

```
cp cmdline.txt /mnt/system-boot/
```

After this, your setup is completed, and you should be able to boot into your system. However, it may be a good idea to take a snapshot beforehand if you need to go back:

```
btrfs subvolume snapshot -r /mnt/new-root/ /mnt/new-root/.snapshots/before-first-boot
```

The -r flag will mark the snapshot as *read-only*.

Unmounting and first boot

After all this steps, our Pi is ready and we will be able to boot into it.

First, we need to unmount our USB disk:

```
umount /mnt/new-root/{var/lib/docker,opt/docker}  
umount /mnt/new-root/{home,root,swap,usr/local,tmp,var/tmp,var/log,/var/snap,/var/mail,/var/spool,.snapshots,}  
umount /mnt/system-boot
```

We could also unmount the image file

```
umount /mnt/old-root  
kpartx -d ubuntu-24.04-preinstalled-server-arm64+raspi.img
```

After that, you can just connect your USB-Disk (our USB-Stick) to your Raspberry Pi. Connect it to power and voilà: You have Ubuntu 24.04, with BTRFS on root, running on your Raspberry Pi 4.

If you want to configure your system a little more **before** the first boot, you should wait until next week, when I publish how to use cloud-init to customize your system.

Final Words

I hope you like this guide and come back for the next one. If you find it useful, I would like to hear from you. Kind words are always appreciated.

If you liked this article, found some mistake or have some comments, write on [Twitter](#) or give a ping on [IRC](#)

The thumbnail for this article is "Raspberry Pi Plastic Case". Original image from Антон Пищулин, CC BY-SA 4.0 <https://creativecommons.org/licenses/by-sa/4.0>, via Wikimedia Commons

1. You can also find them in the btrfs man page: `man 5 btrfs` [↵](#)
2. Otherwise the journal will be updated each time a file is accessed. This causes a lot of I/O, and it's not good for flash memory devices (SSDs, Flash Drives and SD Cards). [↵](#)
3. Fragmentation is specially bad on rotational hard disks. The biggest penalty for flash media is the bigger size of the metadata. The use of autodefrag is a contentious issue. autodefrag isn't well suited for large database workloads. If you have this type of workload, you will need to do a bit of research on the [Btrfs Documentation](#) (and maybe shouldn't be using a pi for it). It also increases the writes on the disk, so not exactly not what we want in an SSD. The default for btrfs is off. You can also defragment the disk manually from time to time. [↵](#)
4. If it's on the root subvolume it would make it impossible to create a snapshot of this subvolume while swap is on. [↵](#)
5. They will also be useful in case we want to do some troubleshooting after a rollback. [↵](#)
6. So we avoid losing them in the event of a rollback. [↵](#)
7. For example: `/var/lib/mariadb` (MariaDB), `/var/lib/mysql` (MySQL), `/var/lib/postgresql` (PostgreSQL), `/var/lib/influxdb` (InfluxDB), etc. [↵](#)
8. Docker will also create subvolumes here, and you want to avoid subvolumes in `@`. [↵](#)
9. All the arguments that we ignored will be taken from the first mounted subvolume (in this case `/`), since they are all in the same device. [↵](#)
10. This guide takes into account these caveats. For more information see Swapfile support on the btrfs docs: <https://btrfs.readthedocs.io/en/latest/Swapfile.html> [↵](#)
11. As you should! Please, do not just copy and paste commands from the internet, specially when running as root. [↵](#)

12. Don't forget to create the mount point for the storage disk: `mkdir /mnt/new-root/mnt/storage` [↩](#)
13. After you carefully read it, of course. [↩](#)